
TD Correction (Invariants) - Correction

Question 1 : Considérons la fonction suivante calculant la somme des n premiers entiers.

```
1 def somme_entiers(n):
2     # pre-condition : n entier naturel
3     s = 0
4     i = 0
5     while i < n:
6         i = i + 1
7         s = s + i
8     return s
```

1. Démontrer que cet algorithme termine.
2. Donner un invariant pour cet algorithme.
3. Prouvez la correction partielle de cet algorithme. En conclure la correction totale.

Correction Q1.

1. **Terminaison** : On peut choisir comme variant de boucle $n - i$ qui reste dans $\mathbb{N}$ car $i < n$ et i augmente de 1 à chaque itération donc $n - i$ diminue de 1 à chaque itération. Donc l'algorithme termine.
2. On choisit l'invariant $I : s = \sum_{k=1}^i k$.
3. **Invariant** :
 - **Initialisation** : Avant la boucle, $s = 0$ et $i = 0$. La somme vide contenant 0 termes vaut bien 0, donc l'invariant est vérifié.
 - **Conservation** : Supposons I vrai au début de l'itération, on a bien $s = \sum_{k=1}^i k$ et $i < n$.
On a alors $i' = i + 1$ et $s' = s + i'$.
Donc $s' = s + i' = \left(\sum_{k=1}^i k\right) + i + 1 = \sum_{k=1}^{i+1} k = \sum_{k=1}^{i'} k$.
L'invariant est donc conservé.
 - **Sortie** : En sortie de boucle, l'invariant $s = \sum_{k=1}^i k$ est vrai et la condition de sortie est fausse, à savoir $i \geq n$.
Or on incrémente i de +1. Donc à la sortie stricte, $i = n$.
Donc à la sortie on a $s = \sum_{k=1}^n k$.

Comme l'algorithme termine et que s vaut la somme des n premiers entiers après la boucle, l'algorithme est correct.

Question 2 : Soit l'algorithme de calcul de la puissance x^n itérative via une boucle classique :

```
1 def puissance_iter(x, n):
2     # Pre-condition : n >= 0
3     res = 1
4     i = 0
5     while i < n:
6         res = res * x
7         i = i + 1
8     return res
```

1. Démontrer que cet algorithme termine.
2. Donner un invariant pour cet algorithme.
3. Prouvez la correction partielle de cet algorithme. En conclure la correction totale.

Correction Q2.

1. **Terminaison** : On peut choisir comme variant de boucle $n - i$ qui reste dans $\mathbb{N}$ car $i < n$ et i augmente de 1 à chaque itération donc $n - i$ diminue de 1 à chaque itération. Donc l'algorithme termine.
2. On choisit l'invariant $I : \text{res} = x^i$.
3. **Invariant** :
 - **Initialisation** : Avant d'entrer dans la boucle $i = 0$ et $\text{res} = 1$. Or $x^0 = 1 = \text{res}$, l'invariant est vrai.
 - **Conservation** : Supposons que $\text{res} = x^i$ et que $i < n$. À la mise à jour, on a $\text{res}' = \text{res} \times x$ et $i' = i + 1$.
On a alors $\text{res}' = x^i \times x = x^{i+1} = x^{i'}$.
L'invariant est donc préservé à la sortie de l'itération.
 - **Sortie** : La boucle se termine quand $i \geq n$. L'évolution fait qu'en sortie on a $i = n$. L'invariant assure alors que $\text{res} = x^n$.

Question 3 : Soit l'algorithme de division entière itérative vu lors du TD sur la terminaison :

```
1 def div_entiere(a, b):
2     # Pre-condition : a >= 0 et b > 0
3     q = 0
4     r = a
5     while r >= b:
6         r = r - b
7         q = q + 1
8     return q, r
```

On souhaite prouver qu'à l'issue de cet algorithme, le couple (q, r) forme bien la division euclidienne de a par b , c'est-à-dire $a = b \times q + r$ avec $0 \leq r < b$.

1. Démontrer que cet algorithme termine.
2. Donner un invariant pour cet algorithme.
3. Prouvez la correction partielle de cet algorithme. En conclure la correction totale.

Correction Q3.

1. **Terminaison** : On peut choisir comme variant de boucle r qui reste dans $\mathbb{N}$ car $r \geq b > 0$ et r diminue de $b > 0$ à chaque itération. Donc l'algorithme termine.
2. On choisit l'invariant $I : a = b \times q + r$.
3. **Invariant** :
 - **Initialisation** : Avant la boucle, $q = 0$ et $r = a$. On évalue l'expression : $b \times 0 + a = a$, ce qui est conforme à la pré-condition. L'invariant est vrai.
 - **Conservation** : Supposons I vrai à l'itération, $a = b \times q + r$ et $r \geq b$. Les variables mutables deviennent $r' = r - b$ et $q' = q + 1$.
Regardons $b \times q' + r' = b \times (q + 1) + (r - b) = b \times q + b + r - b = b \times q + r$.
En appliquant l'hypothèse d'induction, ceci est égal à a . L'invariant est bien conservé.
 - **Sortie** : Par terminaison de la boucle, condition de boucle fausse : on a obligatoirement en sortie que $r < b$. De plus, l'évolution montre que r est resté positif ou nul (il baisse en paliers de b). On a donc un $0 \leq r < b$, couplé à l'invariant terminal $a = b \times q + r$. On ne peut donc plus diviser r par b et obtenir un reste plus petit, donc on a bien la division euclidienne.

Question 4 : Considérons l'algorithme d'exponentiation rapide itérative suivant :

```
1  def expo_rapide_iter(x, n):
2      # Pre-condition : n >= 0
3      res = 1
4      a = x
5      b = n
6      while b > 0:
7          if b % 2 == 1:
8              res = res * a
9          a = a * a
10         b = b // 2
11     return res
```

1. Démontrer que cet algorithme termine.
2. Donner un invariant pour cet algorithme.
3. Prouvez la correction partielle de cet algorithme. En conclure la correction totale.

Correction Q4.

1. **Terminaison** : On peut choisir comme variant de boucle b . Initialement $b = n \geq 0$. À chaque passage dans la boucle strict on a $b > 0$. À la fin de chaque itération, la nouvelle valeur assignée à la variable mutable est $\lfloor \frac{b}{2} \rfloor$. Puisque $b > 0$, on a $0 \leq \lfloor \frac{b}{2} \rfloor < b$. Le variant est bien dans $\mathbb{N}$ et est strictement décroissant, donc l'algorithme termine.
2. On choisit l'invariant $I : \text{res} \times a^b = x^n$.
3. **Invariant** :
 - **Initialisation** : Avant la boucle, on a $\text{res} = 1$, $a = x$ et $b = n$. Ainsi $\text{res} \times a^b = 1 \times x^n = x^n$. L'invariant est vrai à l'entrée de la boucle.
 - **Conservation** : Supposons I vrai à un instant, on a $\text{res} \times a^b = x^n$ et on rentre dans l'itération donc $b > 0$.
Distinguons deux cas selon la parité de b :
 - Si b est pair : $b = 2k$.
Les nouvelles valeurs sont $\text{res}' = \text{res}$, $a' = a^2$ et $b' = \lfloor \frac{b}{2} \rfloor = k$.
Vérifions l'invariant : $\text{res}' \times (a')^{b'} = \text{res} \times (a^2)^k = \text{res} \times a^{2k} = \text{res} \times a^b = x^n$.
 - Si b est impair : $b = 2k + 1$.
Les nouvelles valeurs sont $\text{res}' = \text{res} \times a$, $a' = a^2$ et $b' = \lfloor \frac{b}{2} \rfloor = k$.
Vérifions l'invariant : $\text{res}' \times (a')^{b'} = (\text{res} \times a) \times (a^2)^k = \text{res} \times a \times a^{2k} = \text{res} \times a^{2k+1} = \text{res} \times a^b = x^n$.

Dans tous les cas, on retrouve $\text{res}' \times (a')^{b'} = x^n$. L'invariant est donc conservé à l'issue de l'itération.

- **Sortie** : La boucle termine, donc la condition $b > 0$ devient fausse. Or $b \geq 0$ par construction de l'évaluation du variant, donc $b = 0$.
En injectant dans l'invariant : $\text{res} \times a^0 = x^n \Rightarrow \text{res} \times 1 = x^n \Rightarrow \text{res} = x^n$.
Comme l'algorithme termine, on a donc bien la correction totale, le résultat en sortie valant bien x^n .

Question 5 : Tri par insertion

Soit l'algorithme suivant, qui implémente le tri par insertion :

```
1  def indice_insert(t, x):
2      """Indice où insérer x dans t"""
3      for i in range(len(t)):
4          if t[i] > x:
5              return i
6      return len(t)
7
8  def tri_insertion(t):
9      res = []
10     for j in range(len(t)):
11         i = indice_insert(res, t[j])
12         # insert x dans res à l'indice i
13         res.insert(i, t[j])
14     return res
```

1. Prouver la terminaison de `tri_insertion` à l'aide de variants.
2. Donner un invariant et faites la correction de `indice_insert`.
3. Donner un invariant et faites la correction de `tri_insertion`.
4. En conclure la correction totale.

1.

- Variant de boucle de `indice_insert` (sur la variable i) : $\text{len}(t) - i$
 - Initialisé à $\text{len}(t) - 0 = \text{len}(t)$.
 - Strictement décroissant à chaque itération de i puisque i est augmenté de 1.
 - De plus $\text{len}(t) - i \geq 0$ car $i < \text{len}(t)$.

Donc la boucle termine.

- Variant de boucle de `tri_insertion` : $\text{len}(t) - j$
 - Initialisé à $\text{len}(t) - 0 = \text{len}(t)$.
 - Strictement décroissant à chaque itération de j puisque j est augmenté de 1.
 - De plus $\text{len}(t) - j \geq 0$ car $j < \text{len}(t)$.

Puisque la boucle principale termine et que les appels internes à `indice_insert` terminent aussi, l'algorithme global termine.

2. • Invariant de `indice_insert` :

Au début de chaque itération i , tous les éléments du sous-tableau $t[0 : i]$ sont inférieurs ou égaux à x .

- Preuve de l'invariant :

- Initialisation :

Au début de la première itération, $i = 0$. Le sous-tableau $t[0 : 0]$ est vide, la condition est donc trivialement vérifiée.

- Conservation :

Supposons que l'invariant soit vrai au début d'une itération i . Cela signifie que tous les éléments de $t[0 : i]$ sont $\leq x$.

Lors de l'itération i , le code compare $t[i]$ et x .

- Si $t[i] > x$, la fonction s'arrête et renvoie i .
- Sinon, $t[i] \leq x$. Les éléments de $t[0 : i]$ étant déjà $\leq x$, l'ajout de $t[i]$ implique que tous les éléments de $t[0 : i+1]$ sont $\leq x$.

L'invariant est donc maintenu pour le début de l'itération $i+1$.

- Sortie :

Si la fonction retourne dans la boucle, elle renvoie i . L'invariant assure que $t[0 : i]$ contient des éléments $\leq x$, et on vient de vérifier que $t[i] > x$. L'indice i est donc la bonne position d'insertion pour maintenir le tableau trié.

Si la boucle se termine naturellement, l'invariant garantit que tous les éléments de t sont $\leq x$. La bonne position d'insertion est donc à la fin, soit $\text{len}(t)$, ce que renvoie la fonction. L'algorithme est correct.

3. • Invariant de `tri_insertion` :

Au début de chaque itération, la liste `res` est triée dans l'ordre croissant et contient exactement les j premiers éléments de t .

- Preuve de l'invariant :

- Initialisation :

Avant la première itération, $j = 0$. `res` est une liste vide `[]`. Elle est donc triée et contient bien les 0 premiers éléments de t . L'invariant est vrai.

- Conservation :

Supposons l'invariant vrai au début d'une itération j . `res` est triée et contient les j premiers éléments de t .

Lors de l'itération, on prend l'élément suivant x de t .

La fonction `indice_insert(res, x)` est appelée. Comme `res` est triée, on a prouvé qu'elle renvoie

la position i exacte pour que l'insertion de x conserve res triée.

L'instruction `res.insert(i, x)` place x à cet indice. À la fin de l'étape, res est toujours triée et possède désormais les $k+1$ premiers éléments de t . Cela correspond à l'invariant pour l'itération suivante.

4. On en conclut donc que l'algorithme est totalement correct. Par terminaison de la boucle, lorsque $k = \text{"len"}(t)$, l'invariant indique que la liste res est triée et contient tous les éléments de t . Elle est ensuite renvoyée, ce qui correspond bien au résultat attendu du tri par insertion.

Question 6 : Tri bulle

Soit l'algorithme suivant, qui implémente le tri bulle :

```
1 def tri_bulle(t):
2     n = len(t)
3     for i in range(n):
4         for j in range(0, n - i - 1):
5             if t[j] > t[j + 1]:
6                 t[j], t[j + 1] = t[j + 1], t[j]
7     return t
```

1. Prouver sa terminaison à l'aide de variants.
2. Donner un invariant et faites la correction de la boucle interne.
3. Donner un invariant et faites la correction de la boucle externe.
4. En conclure la correction totale.

Correction Q6.

1. Variant de boucle de la boucle interne : $n - i - j$

- Initialisé à $n - i - 0 = n - i$.
- Strictement décroissant à chaque itération de j puisque j est augmenté de 1.
- De plus $n - i - j \geq 0$ car $j < n - i$.

Donc la boucle termine.

Variant de boucle de la boucle externe : $n - i$

- Initialisé à $n - 0 = n$.
- Strictement décroissant à chaque itération de i puisque i est augmenté de 1.
- De plus $n - i \geq 0$ car $i < n$.

Donc l'algorithme termine.

2. • Invariant de la boucle interne :

À la fin de chaque itération j , l'élément $t[j+1]$ est le maximum du sous-tableau $t[0 : j+1]$.

• Preuve de l'invariant :

► Initialisation :

Lors de la première itération, $j = 0$. Le code compare $t[0]$ et $t[1]$. Si $t[0] > t[1]$, ils sont échangés. À la fin de cette itération, $t[1]$ contient bien la plus grande valeur entre $t[0]$ et $t[1]$. L'invariant est vrai pour $j = 0$.

► Conservation :

Supposons que l'invariant soit vrai à la fin d'une itération $j-1$.

Cela signifie que $t[j]$ est le plus grand élément de $t[0 : j]$.

Lors de l'itération j , le code compare $t[j]$ et $t[j+1]$.

- Si $t[j] > t[j+1]$, il y a échange. Le nouveau $t[j+1]$ est l'ancien $t[j]$, qui était déjà plus grand que tout ce qui le précédait. Il devient donc le maximum de $t[0 : j+1]$.
- Si $t[j] \leq t[j+1]$, il n'y a pas d'échange. Cela signifie que $t[j+1]$ est au moins aussi grand que $t[j]$ (qui est le max de tout ce qui précède). Donc $t[j+1]$ est le maximum de $t[0 : j+1]$.

L'invariant est maintenu à la fin de l'itération j .

3. • Invariant de la boucle externe :

Au début de chaque itération i , le sous-tableau des i derniers éléments, c'est-à-dire $t[n-i:n-1]$, est parfaitement trié, et tous les éléments de ce sous-tableau sont supérieurs ou égaux à tous les éléments du reste du tableau $t[0:n-i-1]$

► Preuve de l'invariant :

– Initialisation :

Au début de la toute première itération, $i = 0$. Le sous-tableau des « 0 derniers éléments » est vide. L'invariant est donc vrai.

– Conservation :

Supposons l'invariant vrai au début de l'itération i . D'après notre analyse de la boucle interne, l'exécution de celle-ci va prendre le plus grand élément du sous-tableau non trié $t[0:n-i-1]$ et le placer à la position $t[n-i-1]$. Il se passe alors deux choses :

- Ce nouvel élément $t[n-i-1]$ est supérieur ou égal à tout ce qui reste dans $t[0:n-i-2]$
- Ce nouvel élément provient de la partie non triée, il est donc inférieur ou égal aux éléments du sous-tableau déjà trié $t[n-i:n-1]$ (selon l'hypothèse de récurrence).

À la fin de l'itération i , la zone triée s'est donc agrandie d'une case vers la gauche. Elle englobe maintenant les $i+1$ derniers éléments, qui sont triés et supérieurs au reste du tableau.

Cela correspond à l'invariant requis pour le début de l'itération suivante ($i+1$).

4. On en conclut donc que l'algorithme est correct et renvoie la liste t triée dans l'ordre croissant.